Week 3 Reflection

Daniyal Shahid Iqbal

January 30, 2026

Optimization aims to identify the "best model" by striking a perfect balance between underfitting and overfitting. Since models can easily overfit during training, we employ regularization techniques such as norm-regularization, dropout, and early stopping. The primary goal of these methods is to discourage model complexity and improve generalization.

Norm-regularization, also known as weight decay, attempts to penalize large weights using a penalty function (such as the L1 or L2 norm). This penalization is added to the loss function, effectively increasing the cost if the model relies on large or numerous weights.

Dropout is another key technique used to prevent overfitting. During training, it randomly sets some activations to zero, effectively ignoring them. This mimics biologically non-firing neurons and forces the model to learn robust features rather than relying on specific pathways. This concept shares a rationale with Data Augmentation, which increases the dataset size for "free" by introducing noise or variations (like changing backgrounds). This helps the model learn to classify based on invariant features rather than trivial static. Finally, we employ early stopping to prevent over-parameterization, halting the training process before the validation loss begins to diverge.

Moving to model architecture, Convolutional Neural Networks (CNNs) represent a significant milestone, often outperforming transformers on smaller datasets. In a traditional fully connected network, a 28×28 image produces a 784-dimensional input vector, requiring the training of approximately 2 million weights. This is inefficient and prone to learning trivial features like background noise.

CNNs, conversely, attempt to learn the spatial structure of the features. We are interested in learning invariant local features—such as the curve of a number "7"—rather than the specific location of pixels. This is achieved via convolution, where a filter (kernel) is applied to patches of the image to find hidden features. For example, applying a filter can help suppress noise to reveal patterns like edges on an image. Different values of the filter create different feature maps such as:

- down sampling
- shifting to left/right
- blurring
- sharpen an image
- edge detection

A typical CNN architecture consists of convolution layers, pooling layers, and a Multi-Layer Perceptron (MLP) for the output. Convolution layers act as feature detectors, where different feature maps are applied to the input vector.

Pooling layers follow convolution to compress the representation. By taking small regions (e.g., 2×2) of each feature map and applying a function like max or average pooling, we reduce the spatial dimensions (width and height).

The filters are applied to the image using a sliding-window approach with stride S. Using a stride of S along each dimension reduces the representation, shrinking it by a factor of approximately S.

For color images, we use a tensor to represent the RGB values. Because a filter reduces the output dimension,

applying convolutions will significantly reduce dimensionality. To prevent this, we use zero padding to maintain the output space at the same dimension as the input space.

The relationship between input size ($R \times R$), filter size ($h \times h$), padding (p), and stride (s) determines the output size:

$$\left(\left\lfloor \frac{R-h+2p}{s} \right\rfloor + 1\right) \times \left(\left\lfloor \frac{R-h+2p}{s} \right\rfloor + 1\right)$$

In summary, paddings increase the output size, while strides decrease the output size. These parameters should be adjusted according to the specific requirements of the task.

Honor Code

I pledge my honor that this lecture summary is my own work and adheres to the guidelines in the instructions.
- Daniyal Shahid Iqbal